

```

#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;

// Merge two sorted halves
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

// Sequential Merge Sort
void sequentialMergeSort(vector<int>& arr, int left, int right) {
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    sequentialMergeSort(arr, left, mid);
    sequentialMergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Parallel Merge Sort using OpenMP sections
void parallelMergeSort(vector<int>& arr, int left, int right, int
depth) {
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    // Limit parallel recursion depth to reduce overhead

```

```

    if (depth <= 0) {
        sequentialMergeSort(arr, left, right);
        return;
    }

    #pragma omp parallel sections
    {
        #pragma omp section
        parallelMergeSort(arr, left, mid, depth - 1);

        #pragma omp section
        parallelMergeSort(arr, mid + 1, right, depth - 1);
    }

    merge(arr, left, mid, right);
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n), arr_copy;

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    arr_copy = arr;

    // Sequential timing
    double start = omp_get_wtime();
    sequentialMergeSort(arr, 0, n - 1);
    double end = omp_get_wtime();

    cout << "\nSequential Sorted Array: ";
    for (int x : arr)
        cout << x << " ";
    cout << "\nTime (Sequential): " << (end - start) << "
seconds\n";

    // Parallel timing
    start = omp_get_wtime();
    parallelMergeSort(arr_copy, 0, n - 1, 3);    // depth can be
adjusted
    end = omp_get_wtime();

    cout << "\nParallel Sorted Array: ";
    for (int x : arr_copy)
        cout << x << " ";
    cout << "\nTime (Parallel): " << (end - start) << " seconds\n";

    return 0;
}

```

